Operatori logici e bitwise

Lezione 2 — 3MAK, SST-INF-01c — Dal valore vero/falso al singolo bit

01/10/2026

1 Ripasso: l'operatore modulo e i blocchi di codice

Esempio

Correzione dell'esercizio della lezione precedente: **dichiarare** due variabili intere, **sommarle**, e verificare se il risultato è **pari o dispari** con l'operatore **modulo**.

```
int a;
int b;
int c = a + b;

if (c % 2 == 1) {
  // c e' dispari
}
```

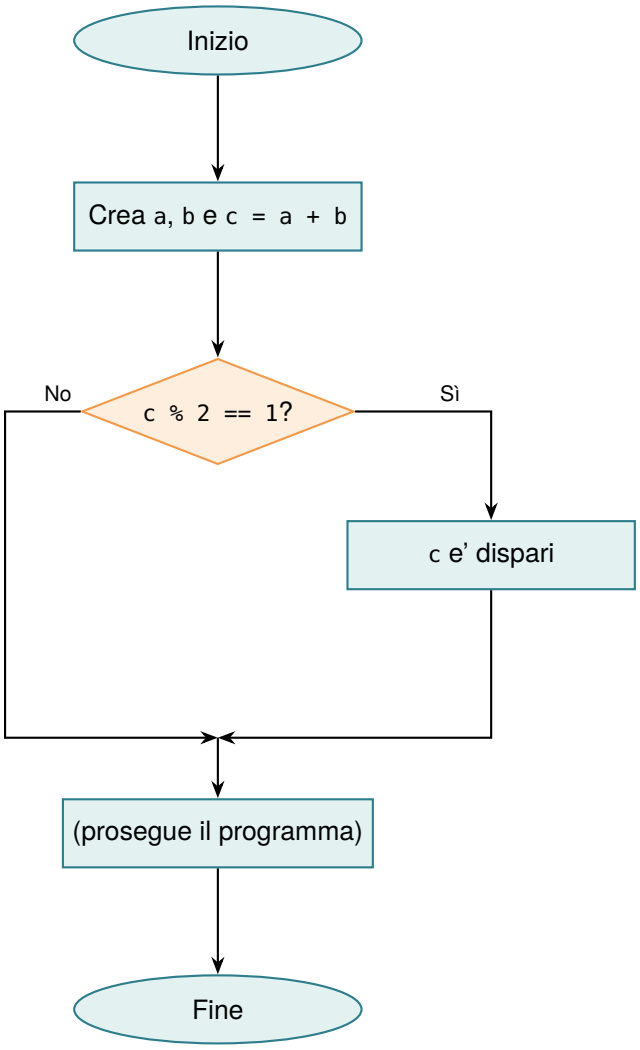


Figura 1: Diagramma di flusso dell'esercizio: verifica se `c = a + b` è dispari.

Attenzione

Un **errore frequente**: assegnare **valori arbitrari** alle variabili (es. `a = 2; b = 4;`) quando l'esercizio non lo richiede. Le **formule** si scrivono con i **nomi delle variabili**, non sostituendo numeri a caso – il codice deve funzionare per **qualunque** valore che `a` e `b` potranno avere, non solo per l'esempio che si ha in mente.

Nota

In classe, correggendo l'esercizio, un allievo ha inizialmente confuso il simbolo **modulo** (`%`) con quello di **divisione** (`/`): se fossero la stessa cosa, non avrebbe senso avere due simboli diversi nel linguaggio. Un primo tentativo di definizione ("il modulo calcola il resto") non bastava a spiegare perché, ad esempio, non abbia senso calcolare "3,2 modulo 3,2": il **resto** di una divisione normale tra numeri con la virgola **non è ben definito** come lo è tra interi (non si può dire "3,2 diviso 2 fa resto di 1,2" – un resto deve sempre essere un **numero intero** più piccolo del divisore). Da qui la **definizione precisa**:

Definizione

L'operatore **modulo** (`%`) calcola il **resto** della divisione **tra interi**. **Non è definito** per numeri con la virgola: non ha senso chiedersi "quanto fa 3,2 modulo 3,2", perché il modulo si applica **solo a valori interi** con un resto ben definito.

Esempio

`2 % 10` vale `2`: la **divisione intera** $2/10$ fa 0, con **resto** 2.

Definizione

In C (e nella maggior parte dei linguaggi di programmazione) le **graffe** `{ }` raggruppano più **istruzioni** e le trattano come un **blocco unico**. Al **compilatore** non importa come il codice viene **formattato** (spazi, a capo, indentazione): l'unica cosa che conta è il **blocco corretto** delimitato dalle graffe. La formattazione serve esclusivamente alla **leggibilità** per chi scrive e legge il codice.

Esempio

Un'**analogia**: nei **videogiochi** più comodi, un singolo tasto può essere programmato per eseguire un'intera **sequenza di mosse** (es. "giù giù destra sinistra calcio pugno") come se fosse un'unica pressione. Le **graffe** in C funzionano allo stesso modo: raggruppano una **sequenza di istruzioni** e la trattano come un blocco unico.

Nota

Buone pratiche di scrittura del codice:

- Si apre la graffa e si richiude **in verticale, sulla stessa colonna** di dove inizia il blocco.
- **Indentazione**: il contenuto di un blocco si scrive **rientrato** (una tabulazione) rispetto al bordo, per rendere visibile a colpo d'occhio cosa **appartiene** a quel blocco.
- **Un'istruzione per riga**: si potrebbero scrivere più istruzioni sulla stessa riga (al compilatore non importa), ma un programmatore che scrive bene ne scrive **sempre una per riga**.
- Il **punto e virgola** si scrive **solo** dopo le singole istruzioni dentro il blocco, **mai** dopo la graffa di chiusura di un costrutto come `if`.

Esempio

Se il blocco contiene una sola istruzione, le graffe si possono anche **omettere** – ma è buona abitudine **metterle sempre**, perché capiterà spesso che il blocco **cresca** a più righe:

```
if (c % 2 == 0)
  pari;          // valido, ma solo per una singola istruzione

if (c % 2 == 0) {
```

```
  pari;          // preferibile: sempre con le graffe
}
```

2 Ripasso: i tipi di dato

Nota

`int`, `float` e `char` si differenziano per come viene gestita la **dimensione** e il **tipo** di numero che contengono. **`float`** rappresenta numeri **con la virgola**; `int` e `char` no. Un tipo più grande serve **solo se** il numero da contenere è effettivamente grande: usare un `int` per contenere un numero che sta comodamente in un `char` (es. il numero di allievi di una classe, che può arrivare a 255 con un `char`) è uno **spreco di spazio** in memoria, esattamente come occupare un grosso scatolone dell'armadio per riporre solo tre paia di calzini.

3 Gli operatori logici "a valore"

Definizione

Gli operatori logici **AND** (`&&`), **OR** (`||`) e **NOT** (`!`) già visti valutano il **valore complessivo** della variabile: in C, solo il **numero 0** è **falso**, **qualunque altro valore** (anche negativo) è considerato **vero**.

Esempio

```c
int a = 5;
int b = 10;
int c = a && b;
```

`a` vale 5 (diverso da 0, quindi **vero**), `b` vale 10 (vero). `vero AND vero` dà **`vero`**: formalmente il risultato potrebbe essere un qualunque valore diverso da 0, ma la **convenzione** quasi universale dei compilatori è restituire **`1`**. Quindi `c` vale `1`.

Esempio

Con gli stessi valori (`a = 5`, `b = 10`), `c = a || b` vale `1`: basta che **almeno una** delle due condizioni sia vera. In questo caso particolare AND e OR danno lo stesso risultato, ma **non è sempre così** – dipende dai valori.

Esempio

```c
int a = 5;
int c = !a;   // c vale 0
```

`a` vale 5, quindi è vero. Il **punto esclamativo** (`NOT`) **inverte**: `NOT vero` = falso, cioè, in valore

numerico, `0`.

4 Gli operatori bitwise: AND, OR, NOT bit a bit

Definizione

Esiste una seconda categoria di **operatori logici**, detti **bitwise** (letteralmente "a livello di bit"): `&` (AND bitwise), `|` (OR bitwise), `~` (NOT bitwise, tilde). Sono scritti con un **singolo** simbolo, a differenza dei corrispondenti operatori "a valore" (`&&`, `||`) che usano il simbolo **doppio**. Sono fondamentali per chi scrive **firmware** ed **embedded**, perché permettono di **manipolare singoli bit** di un registro – ad esempio i **piedini** di un microcontrollore.

Attenzione

Un operatore "a valore" valuta la variabile **nel suo complesso** (vero/falso); un operatore **bitwise** esegue l'operazione **bit per bit**, confrontando le **cifre binarie** una per una, in verticale, nella **stessa posizione**. Il risultato non è più semplicemente vero/falso, ma un **numero** vero e proprio.

4.1 Conversione decimale → binario

Definizione

Per scrivere un numero **decimale** in **binario**, si **divide** ripetutamente per 2 e si annotano i **resti**; i resti letti **dal basso verso l'alto** (dall'ultimo calcolato al primo) formano il numero binario.

Esempio

Conversione di **10** in binario:

- $10 \div 2 = 5$ resto 0
- $5 \div 2 = 2$ resto 1
- $2 \div 2 = 1$ resto 0
- $1 \div 2 = 0$ resto 1

Leggendo i resti **dal basso verso l'alto**: `1010`. Su 8 bit: `00001010`.

Esempio

Conversione di **175** in binario (dividendo ripetutamente per 2):

- $175 \div 2 = 87$ resto 1
- $87 \div 2 = 43$ resto 1
- $43 \div 2 = 21$ resto 1

- $21 \div 2 = 10$ resto 1
- $10 \div 2 = 5$ resto 0
- $5 \div 2 = 2$ resto 1
- $2 \div 2 = 1$ resto 0
- $1 \div 2 = 0$ resto 1

Leggendo i resti dal basso verso l'alto: `10101111`.

4.2 Conversione binario → decimale

Definizione

Per convertire un numero binario in decimale si usa la **notazione posizionale**: il bit **più a destra** vale 2^0, il successivo 2^1, poi 2^2, e così via. Si **somma** 2^n per ogni posizione dove il bit **vale 1** (dove il bit vale 0, quella posizione **non contribuisce** alla somma).

Esempio

Il numero binario `10010100` si converte così: i **bit a 1** sono nelle posizioni 2, 4 e 7 (contando da destra, a partire da 0).

$$2^2 + 2^4 + 2^7 = 4 + 16 + 128 = 148$$

Esempio

Il numero binario `00110011` si converte così: bit a 1 nelle posizioni 0, 1, 4, 5.

$$2^0 + 2^1 + 2^4 + 2^5 = 1 + 2 + 16 + 32 = 51$$

4.3 L'AND bitwise

Esempio

Confronto diretto con l'AND "a valore" visto prima, usando gli stessi numeri `a = 5` e `b = 10`:

```
int a = 5;
int b = 10;
int c = a & b;   // AND bitwise, simbolo singolo
```

Si scrivono i due numeri in binario su 8 bit:

$$5 = 00000101 \qquad 10 = 00001010$$

Bit per bit, non c'è **mai** una colonna in cui entrambi valgono 1:

$$\begin{array}{r} 00000101 \\ \text{AND } 00001010 \\ \hline 00000000 \end{array}$$

Il risultato è `00000000`, cioè **0** (falso): un esempio concreto di come l'AND "a valore" (`a && b`, che con questi stessi numeri valeva vero/1) e l'AND bitwise (`a & b`) possano dare **esiti completamente diversi**, perché lavorano su **livelli diversi** – il primo sul **valore complessivo**, il secondo **bit per bit**.

Esempio

```
int a = 6;
int b = 7;
int c = a & b;   // AND bitwise, simbolo singolo
```

Si scrivono i due numeri in binario su 8 bit:

$$6 = 00000110 \qquad 7 = 00000111$$

L'AND si applica **bit per bit**, **colonna per colonna** (vale 1 **solo dove entrambi** i bit sono 1):

$$\begin{array}{r} 00000110 \\ \text{AND } 00000111 \\ \hline 00000110 \end{array}$$

Il risultato è `00000110`, cioè **ancora 6**. Quindi `c` vale `6`.

4.4 L'OR bitwise

Esempio

Con gli stessi numeri (`0000 0110` e `0000 0111`), l'**OR bitwise** vale 1 dove **almeno uno** dei due bit è 1:

$$\begin{array}{r} 00000110 \\ \text{OR } 00000111 \\ \hline 00000111 \end{array}$$

Esempio

Un secondo esempio di OR bitwise, con due numeri in cui i bit a 1 occupano **zone diverse**: `0000 0100` (il numero 4) e `0000 1011` (il numero 11). L'OR bit a bit dà `0000 1111`, che convertito in decimale vale:

$$2^0 + 2^1 + 2^2 + 2^3 = 1 + 2 + 4 + 8 = 15$$

Attenzione

L'AND bitwise viene chiamato anche "**prodotto logico**" e l'OR bitwise "**somma logica**" – ma questi nomi sono solo un'**analogia concettuale**, **non** un'**operazione aritmetica** reale. Non si deve **mai** pensare che l'OR bitwise tra due numeri corrisponda alla loro **somma decimale**: 10 `OR` 5 vale 1010 `OR` $0101 = 1111 = 15$, che per puro caso coincide con $10 + 5$.
In classe, a questo punto, è stato chiesto "quanto fa 11 `OR` 5?": una prima risposta data a voce è stata scartata perché non coerente con il **calcolo bit a bit**. Il risultato corretto si ottiene **sempre** scrivendo i due numeri in **binario** e applicando l'OR **colonna per colonna**: 11 `OR` 5 vale

1011 `OR` $0101 = 1111 = 15$, e **non** 16 come darebbe la somma aritmetica (che qui, a differenza del caso precedente, non coincide per puro caso con il risultato bitwise). Si stanno sempre **combinando i singoli bit** nella stessa posizione, non i valori decimali complessivi – è proprio questo il motivo per cui affidarsi alla "somma" come **scorciatoia** può portare a una **risposta sbagliata**.

4.5 Il NOT bitwise

Definizione

Il NOT bitwise (~, tilde) **inverte ogni singolo bit** del numero: ogni 0 diventa 1 e ogni 1 diventa 0.

Esempio

```
int x = 42;
int y = ~x;   // NOT bitwise
```

Si converte 42 in binario:

- $42 \div 2 = 21$ resto 0
- $21 \div 2 = 10$ resto 1
- $10 \div 2 = 5$ resto 0
- $5 \div 2 = 2$ resto 1
- $2 \div 2 = 1$ resto 0
- $1 \div 2 = 0$ resto 1

Quindi $42 =$ `00101010`. **Invertendo** ogni bit: `11010101`. Convertendo in decimale (bit a 1 nelle posizioni 0, 2, 4, 6, 7):

$$2^0 + 2^2 + 2^4 + 2^6 + 2^7 = 1 + 4 + 16 + 64 + 128 = 213$$

Quindi `y` vale **`213`**.

Nota

Un modo **alternativo** (e più rapido) per capire il risultato del NOT bitwise su un numero a 8 bit: i numeri binari a 8 bit si comportano come un "**cerchio**" che va da 0 a 255 e poi torna a 0 (perché sono rappresentati con un **numero finito** di bit). Invertire tutti i bit di un numero n equivale a **calcolare** $255 - n$: infatti $255 - 42 = 213$, lo stesso risultato ottenuto bit per bit.

5 Perché servono gli operatori bitwise

Esempio

Un esempio concreto dell'utilità dell'AND "a valore" nella **sicurezza industriale**: nelle **presse** industriali, un tempo capitava che un operatore **si ferisse** perché, per far scendere la pressa, appoggiava una mano sulla macchina mentre frenava con l'altra. La soluzione: si installano **due pulsanti** distanti tra loro, e la pressa si attiva solo se **entrambi** sono premuti contemporaneamente:

```
if (pulsante_A && pulsante_B) {
  // la pressa puo' scendere
}
```

Con una singola condizione AND, si garantisce che l'operatore debba avere **entrambe le mani** occupate sui pulsanti, **lontano** dalla **zona di pericolo**.

Definizione

Nei microcontrollori più semplici (diversi da Arduino) **non esistono** funzioni pronte come **`digitalWrite`**/**`digitalRead`**: per accendere, spegnere o leggere un singolo **piedino** bisogna agire **direttamente**, con operatori **bitwise**, sul **registro** che rappresenta quel gruppo di piedini – ad esempio **impostando a 1** un singolo bit con un OR bitwise, o **azzerandolo** con un AND bitwise mascherato da un NOT.

Nota

Perché non usare semplicemente `digitalWrite` come su Arduino? Internamente, `digitalWrite` esegue **molte operazioni aggiuntive** prima di agire sul singolo pin, per garantire sicurezza e **compatibilità** generica. Questo richiede più **tempo macchina**. Agendo **direttamente sul registro** con un singolo operatore bitwise, lo stesso risultato si ottiene con un'istruzione **molto più rapida**: la macchina lavora più velocemente, e a parità di compito si può usare un **processore più lento** e quindi più **economico**. È uno dei motivi per cui, in ambito industriale, un prodotto commerciale non userà **mai** una scheda Arduino così com'è: le board amatoriali sono ottime per imparare, ma l'approccio "a **basso livello**" coi registri resta la **scelta professionale**, dove le **prestazioni** e il **costo** hardware contano.

6 Cosa ricordare di questa lezione

- Il **modulo** (`%`) calcola il **resto** della divisione tra **interi**: **non è definito** per numeri con la virgola.
- Le **graffe** `{ }` raggruppano istruzioni in un **blocco unico**; l'**indentazione** e un'istruzione per riga servono solo alla **leggibilità**, **non cambiano** il comportamento del programma.
- Gli operatori logici **a valore** (`&&`, `||`, `!`) valutano l'**intera variabile**: in C, **0 è falso**, **tutto il resto** è vero.
- Gli operatori **bitwise** (`&`, `|`, `~`, simbolo singolo) operano **bit per bit**: servono a **manipolare**

singoli pin/bit di un **registro**, fondamentali in firmware/embedded.

- Per convertire decimale $\rightarrow$ binario: **dividere** ripetutamente per 2, leggere i resti **dal basso verso l'alto**. Per convertire binario $\rightarrow$ decimale: **somma delle potenze di 2** nelle posizioni dove il bit vale 1.
- **Attenzione**: AND/OR bitwise ("prodotto/somma logica") non sono **mai** una vera moltiplicazione/addizione aritmetica – si combinano sempre i **singoli bit**, non i valori decimali.